

Nombre y apellido: Kreszczuk Oscar Mauricio

Carrera: Licenciatura en Ciencia de Datos

[Collab](#)

[Dataset \(drive\)](#)

Dataset

Para el proyecto se eligió el *The Simpsons Characters Dataset* (de Alex Attia, disponible en Kaggle). Cuenta originalmente con más de 42.000 capturas de pantalla tomadas directo de los capítulos de la serie. Los datos se rigen bajo la licencia **Creative Commons CC BY-NC-SA 4.0** (permite su uso libre para fines educativos e investigación, siempre que se mencione al autor, no se comercialice y se comparta bajo las mismas reglas).

El dataset incluye un archivo clave llamado `annotation.txt`, que guarda las coordenadas absolutas de las cajas de límites (bounding boxes) de los personajes presentes en las escenas con el formato estándar: **Ruta, x1, y1, x2, y2, Clase**.



Trabajo previo con los datos

Limpieza y formateo

Primero estandarizamos las imágenes: las pasamos a RGB, las redimensionamos a un tamaño único y dividimos el valor de los píxeles por 255. De esta forma, normalizamos todo al rango de 0 a 1 para ayudar a que la red pueda entrenar mejor. Además, como también se quería realizar detección, se usó el archivo `annotations.txt` del dataset para extraer las coordenadas de las Bounding Boxes. A estas coordenadas también las normalizamos dividiéndolas por el ancho y alto real de cada foto, para que la resolución original no afecte a los cálculos.

Balanceo

Al explorar el dataset, se encontró un problema: estaba muy desbalanceado. Había personajes con 2.000 capturas y otros con apenas 400. Si le proporcionábamos todas las capturas a la red, iba a generar un sesgo y aprender a predecir siempre a los personajes que más aparecían (además que tardaría demasiado tiempo). Para solucionarlo, se decidió usar solo 6 clases principales (homer_simpson, bart_simpson, lisa_simpson, marge_simpson, ned_flanders y montgomery_burns) y "cortar" la cantidad a exactamente 300 imágenes por personaje. Así habrá un conjunto perfectamente balanceado de 1.800 imágenes en total.

Partición en Train y Test

Se dividió estas 1.800 imágenes en dos grupos: un 80% para entrenar a los modelos (1.440 imágenes) y se guarda el 20% restante para testarlos (360 imágenes que la red nunca vio). Un detalle clave acá fue usar la técnica de estratificación (stratify) al hacer la división. Esto garantiza que la proporción de los 6 personajes fuera exactamente la misma en ambos conjuntos, evitando cualquier alteración engañosa.

Desafíos

Fondos muy cargados: No son fondos simples que se sean siempre los mismos. Los personajes aparecen en la taberna de Moe, la planta nuclear o la escuela (entre otros muchos más), rodeados de objetos y detalles. Este ruido visual confunde a los modelos, que deben aprender a ignorar el entorno para enfocarse en el personaje.

Cambios de pose y gestos: Las imágenes muestran a los personajes de frente, de perfil, de tres cuartos, gritando, enojados o deformados según la escena.

Proximidad cromática: Los personajes comparten exactamente el mismo color amarillo para la piel. Como no se puede guiar por el color como una característica diferenciadora, esto fuerza a los modelos a depender de la estructura espacial de las capturas (como el pelo estrella de Lisa, la verticalidad del pelo de Marge o los anteojos de Flanders) para lograr una separación precisa de las clases.



Procesamiento Digital de Imágenes (PDI)

Transformaciones de intensidad

Conversión a escala de grises: Se eliminaron los canales de color para calcular la intensidad lumínica de cada píxel. Al sacar el color, se vio que los elementos clave (las líneas negras de los contornos, los ojos y las siluetas) se mantienen intactos. Sin embargo, para los modelos de redes neuronales esto no sirve: al unificar los canales, personajes con cuerpos parecidos o fondos similares se vuelven muy difíciles de diferenciar. Conviene dejar el formato RGB para usar el color y la forma al mismo tiempo.

Ecualización de histograma: Se aplicó este método sobre la imagen en grises para estirar el contraste. Al probarlo en una captura oscura (Homero en la taberna de Moe), el cuerpo del personaje se aclaró un montón, pero aparecieron dos problemas graves: se perdieron los detalles en las zonas claras (la camisa blanca se volvió muy lisa y detalles de las líneas del dibujo) y apareció un granulado (pixelado) en los degradados suaves del techo (y otros detalles sutiles). Para el procesamiento clásico ayuda a ver cosas en la sombra, pero para meter en una red es contraproducente porque deforma las texturas reales.



Muestreo y degradación

Baja resolución (Blurring):

Se simuló una pérdida de resolución bajando la imagen de 128 x 128 a un tamaño de 32x32 píxeles, para después volver a estirla al tamaño original mediante interpolación. El resultado mostró una pérdida total de información: los detalles chicos (como la separación de los dientes o las líneas finas de los ojos) desaparecieron en la reducción y la matemática no pudo recuperarlos, dejando los bordes borrosos. Aunque trabajar en 32x32 haría que los modelos corran más rápido, destruir la geometría de esta forma perjudica la extracción de características.

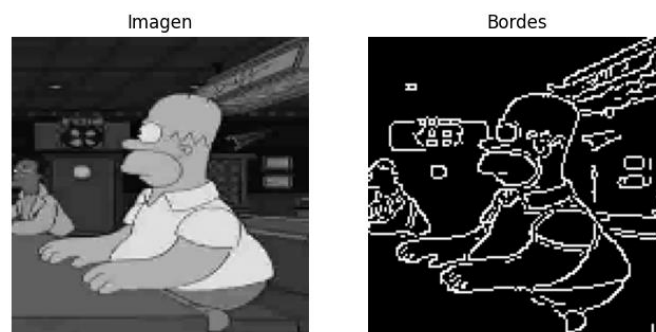
Agregado de ruido: Se le sumó ruido gaussiano a la foto para ver qué tan tolerante era el dibujo. El resultado visual fue un granulado en toda la captura. Pero, a pesar del desorden visual, los ojos de Homero y las líneas negras principales se seguían distinguiendo bien a simple vista. Esto pasa porque el estilo de animación tiene bordes muy marcados y resiste un poco

mejor el ruido que una foto real.



Detección de bordes

Algoritmo de Canny: Se corrió el filtro de Canny sobre la imagen de grises. El algoritmo aisló muy bien los saltos bruscos de luz y dibujó a la perfección los contornos de Homero y del fondo (las líneas del billar, las ventanas). Sirve para trabajar solo con las formas y ahorrar procesamiento. Pero para entrenar una red neuronal es una mala idea: elimina datos valiosos como los bloques de color y, si la imagen llega a tener un mínimo de ruido, Canny detectaría cada punto de el como si fuera un borde real, llenando la pantalla de basura visual.



Clasificación

ANN

El primer experimento consistió en aplanar las imágenes para convertirlas en un vector unidimensional y conectarlo a una red densa totalmente conectada (Sequential con capas Dense y Dropout al 30%).

Resultado: El rendimiento fue nulo, registrando un accuracy estancado en **16.67%** (el equivalente exacto al azar en un problema de 6 clases). La pérdida se congeló en 1.7918 desde la primera época y la matriz de confusión demostró que la red desarrolló un sesgo absoluto hacia la primera clase (homer_simpson), fallando en el 100% de los demás personajes.

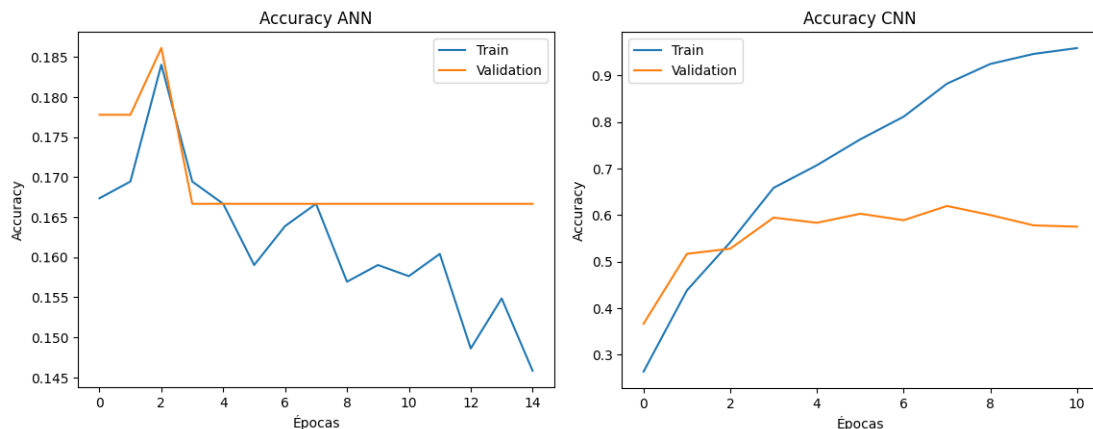
Al "aplanar" la matriz al inicio, se destruyó la relación de vecindad entre píxeles (la estructura espacial). La red pierde la noción de contornos y formas locales. Si un rasgo cambia levemente, la capa densa lo interpreta como una entrada inconexa, provocando underfitting severo.

CNN

Para solucionar el problema del aplanado, se construyó una CNN desde cero con tres bloques consecutivos de capas Conv2D (3 x 3) y MaxPooling2D (2x2), aplicando el aplanado recién al final antes de las capas densas de clasificación. Se usó Early Stopping con una paciencia de 5 épocas.

Resultado: El modelo alcanzó un accuracy en validación de entre el 70% y 73%. Sin embargo, el entrenamiento se cortó prematuramente en la época 11 debido a que la pérdida de validación dejó de mejorar en la época 6.

Pero hubo un problema claro de overfitting, a partir de la tercera época, la curva de entrenamiento siguió subiendo hacia la perfección, pero la de validación se planchó. Al contar con un dataset chico (1440 imágenes), la red propia terminó memorizando en lugar de aprender los rasgos de los personajes, perdiendo la capacidad de generalizar.

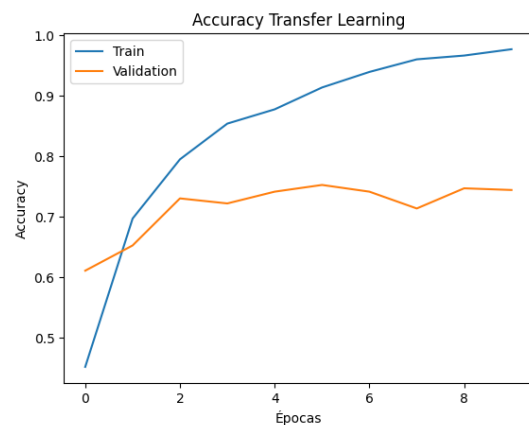


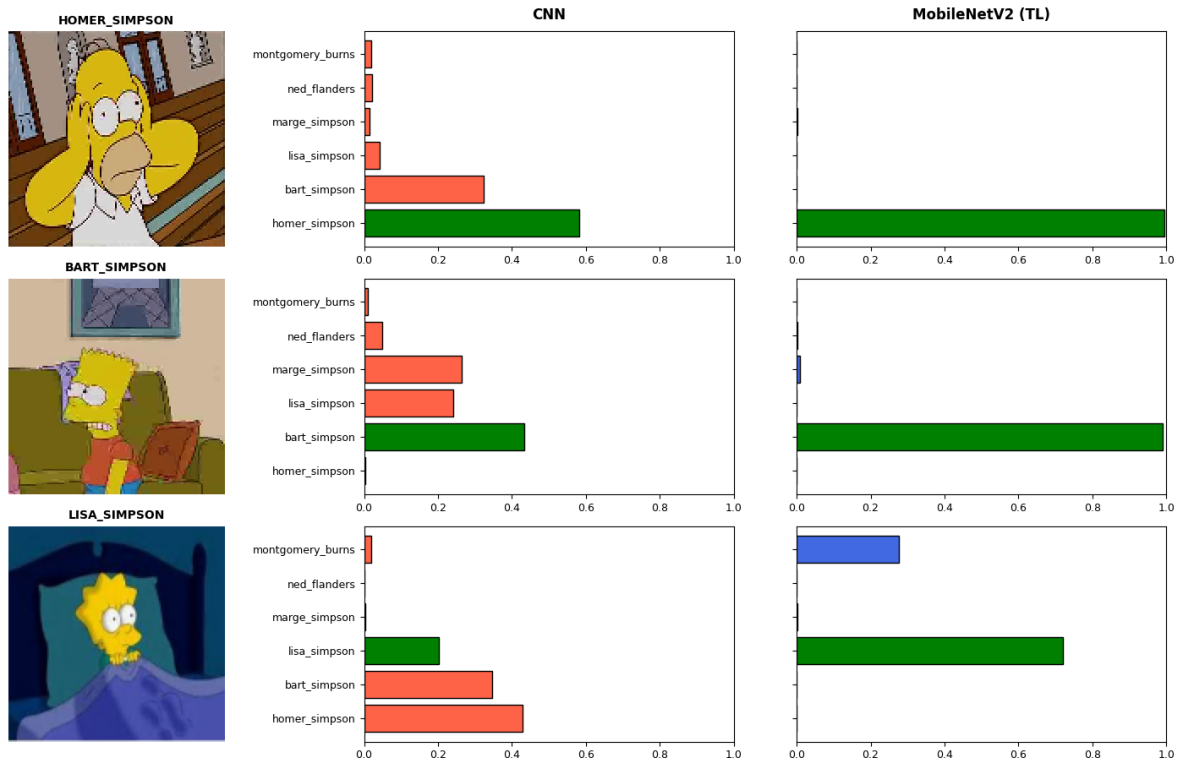
Transfer Learning (MobileNetV2)

Para contrarrestar el overfitting, se implementó transfer learning congelando la base convolucional de **MobileNetV2** (preentrenada con millones de imágenes reales en *ImageNet*) y añadiendo una capa densa adaptada a las 6 clases.

Resultado: Fue la solución más robusta para clasificación pura, estabilizando el accuracy de validación en un **74% - 75%** y manteniendo la pérdida controlada (entre 0.76 y 0.85).

Al reutilizar un extractor de características preentrenado, el modelo ya sabe identificar contornos, curvas y contrastes geométricos básicos. No necesita gastar datos en aprender a ver; solo adapta esas estructuras a las formas de los personajes, mitigando el impacto de los fondos.



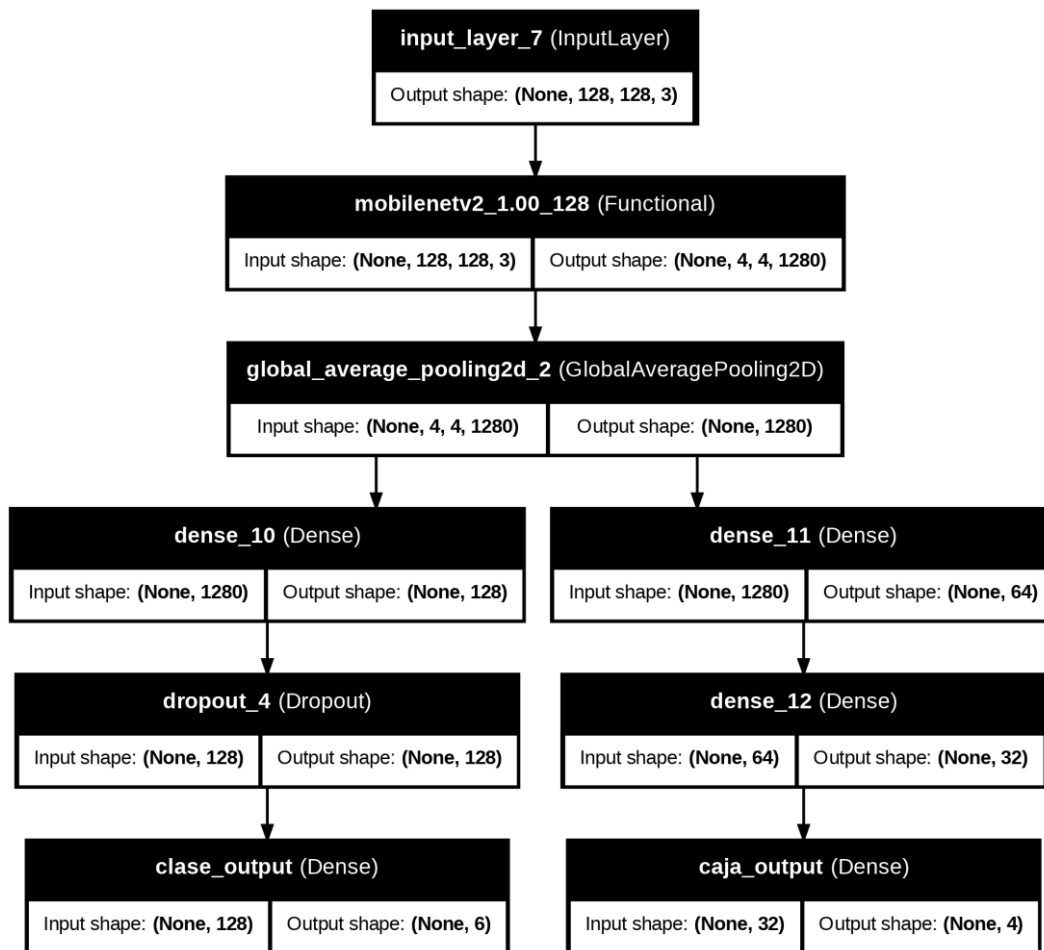


Detección y Modelo Multi-Output

Queriendo implementar detección, se intentó utilizar modelos de ello ya entrenados, pero los resultados no resultaron convincentes. Recordando que el dataset original proveía el archivo annotations.txt con las coordenadas exactas de las cajas delimitadoras (a utilizar para entrenar la detección), se optó por volver al Transfer Learning para la parte de convolución. Se diseñó las capas de convolución con MobileNetV2 (congelada) como extractor de características, pero bifurcando la salida en dos ramas independientes:

Rama de Clasificación: Una capa densa de 128 neuronas (con ReLU y un *Dropout* al 30%) que termina en una salida de 6 neuronas con activación **Softmax** para dar las probabilidades de los personajes.

Rama de Regresión: Dos capas densas consecutivas (de 64 y 32 neuronas) que terminan en una salida de 4 neuronas con activación **Sigmoide**, encargada de entregar las 4 coordenadas normalizadas de la caja.



Como las dos ramas miden errores con criterios distintos, se configuraron pérdidas separadas: Sparse Categorical Crossentropy para la identidad del personaje y el Error Cuadrático Medio (MSE) para las coordenadas de la caja.

Debido a que los errores se calculan en magnitudes diferentes y las tareas tienen distintas velocidades de convergencia, se configuró un esquema de pesos ponderados en la compilación del modelo:

$$\mathbf{Loss}_{\text{Total}} = 1.0 \times \mathbf{Loss}_{\text{Clase}} + 2.0 \times \mathbf{Loss}_{\text{Caja}}$$

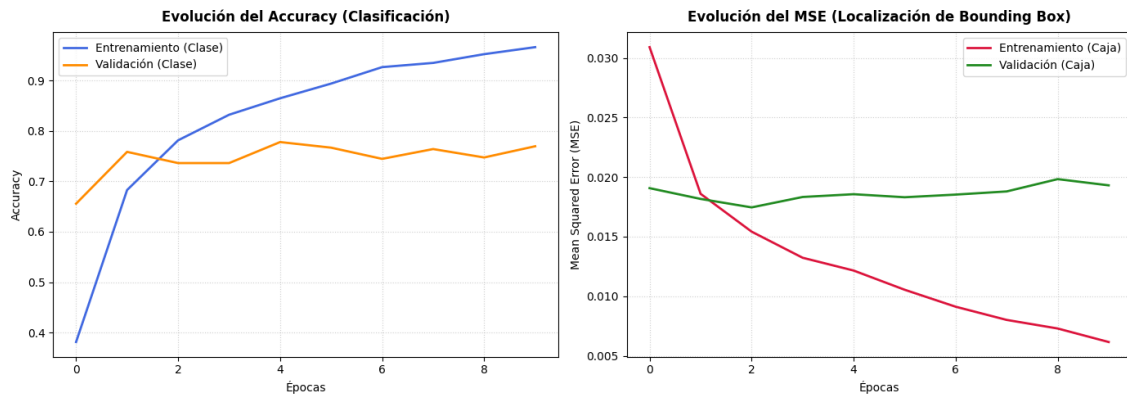
Se decidió darle el doble de prioridad a la pérdida de la caja porque aprender coordenadas desde cero es una tarea mucho más compleja que la clasificación (que ya viene simplificada por los filtros pre-entrenados de MobileNetV2). El entrenamiento usó un Early Stopping con paciencia de 4 épocas, cortando el proceso en la época 10 al detectar que el punto óptimo ocurrió en la época 6.

Ejemplos:



Clasificación (Accuracy): En la rama clasificatoria, el accuracy de validación arrancó alto (65%) pero se terminó estancando entre el 73% y el 77%.

Detección (MSE): El gráfico de evolución mostró que la localización funcionó muy bien. El MSE de validación cruzó las curvas rápidamente y se planchó de forma estable en torno a 0.018, demostrando que el modelo aprendió a encuadrar bien a los personajes sin importar el fondo.



Al obligar al modelo a resolver dos tareas al mismo tiempo bajo una pérdida compartida que prioriza la detección, el algoritmo de Backpropagation genera una competencia por los recursos en las capas compartidas. Resultando en una precisión ligeramente menor para la clasificación.

Conclusiones

1. **Filtros manuales:** Las técnicas clásicas de PDI son excelentes para entender el comportamiento de los píxeles y la distribución de intensidades, lo que nos ayuda a realizar diferentes análisis nosotros mismos.
Al forzar el contraste o aplicar filtros manuales, muchas veces borramos información cromática clave o destruimos texturas por saturación, por lo que en general, no es recomendable aplicarlos antes de que entren las imágenes a los modelos de redes neuronales.
2. **Dataset:** El balanceo manual a 300 fotos por personaje evitó que los modelos se sesgaran hacia las clases mayoritarias. Además, el hecho de que todos los personajes compartan la piel amarilla obligó a los modelos a centrarse en las características espaciales que diferencian a cada uno.
3. **Clasificación:** Diseñar redes densas (ANN) para clasificar imágenes directamente es un error, se aplanan las imágenes lo cual rompe la estructura del espacio y no logrará realizar la clasificación. Por otro lado, entrenar redes convolucionales (CNN) con pocas imágenes lleva directo al overfitting, donde el modelo memoriza las imágenes, pero no puede generalizar. Ante esto, el Transfer Learning con MobileNetV2 demostró ser mejor opción para la clasificación, aportando filtros estables que ignoran el ruido del entorno.